React Quiz 4

Sign in to Google to save your progress. Learn more

Quiz

What is the purpose of **useMemo** in React? 1 point

- ○ It memoizes a function to prevent unnecessary re-rendering of components.
- ○ It prevents the component from re-rendering by ignoring prop changes.
- ○ It memoizes a calculation or expensive operation and only recalculates when the dependencies change.
- ◉ It memoizes all state changes in a component.

Clear selection

What is the best approach to **optimize rendering lists** of components in React? 1 point

- ○ Use React.memo for each item in the list.
- ○ Avoid using keys in list items to prevent React from tracking them.
- ○ Use useState inside each list item to track its own state.
- ◉ Use stable and unique keys for list items and memoize the entire list with useMemo when applicable.

Clear selection

What is the primary purpose of the React Compiler? 1 point

- ○ To transpile JSX into JavaScript using Babel
- ◉ To automatically optimize React components by memoizing values and functions
- ○ To bundle React applications for production
- ○ To convert React code into WebAssembly

Clear selection

Which of the following statements about React Server Components (RSC) is correct? 1 point

- ○ React Server Components can use hooks like useState and useEffect.
- ○ React Server Components are rendered on the client side to improve interactivity.
- ◉ React Server Components can fetch data on the server without exposing sensitive data to the client.
- ○ React Server Components require 'use client' at the top of the file to function correctly.

Clear selection

How does React achieve efficient updates to the user interface? 1 point

- ○ By reloading the entire page on every change
- ◉ By using the Virtual DOM to determine the minimal number of DOM updates
- ○ By using server-side rendering exclusively
- ○ By manually updating the DOM using JavaScript

Clear selection

What is the **primary purpose** of using the **React Context** API? 1 point

- ○ To fetch data from an external API.
- ○ To handle side effects in functional components.
- ◉ To manage global state and pass data deeply through the component tree without manually passing props at every level.
- ○ To improve performance by memoizing functions.

Clear selection

You are building a **complex form** where the state includes multiple fields like name, email, and password. Additionally, you need to track form submission status (e.g., loading, success, or error). Which hook is more suitable for managing the form state and why? 1 point

- ○ useState, because it is simpler and sufficient for managing multiple fields.
- ◉ useReducer, because it provides a more structured way to handle complex state updates and multiple types of actions.
- ○ useContext, because it allows sharing form state between components.
- ○ useMemo, because it allows memoizing the form data.

Clear selection

You are building a **weather application** that fetches data from an API. The data should be fetched only once when the component mounts, and you want to show a loading spinner while the data is being fetched. Which approach will work best for this scenario? 1 point

- (selected) Use useState to store the data and useEffect with an empty dependency array to fetch the data when the component mounts.
- Use useState to store the data and useEffect with the dependency array containing the API URL to fetch the data every time the component re-renders.
- Use useEffect with useRef to ensure the API call is made only once.
- Use useReducer to fetch the data and manually handle state changes.

Clear selection

You are building a **todo application** with Redux to manage the global state. The TodoList component fetches the list of todos from the Redux store and the AddTodo component dispatches actions to add new todos. Which of the following is the **best approach** to handle this? 1 point

- (selected) Use the useDispatch hook in TodoList to fetch todos from the store and useSelector in AddTodo to add todos.
- Use the useSelector hook in TodoList to get the todos from the store, and use the useDispatch hook in AddTodo to dispatch the action to add new todos.
- Use connect() to map state and dispatch to props in both components.
- Use useContext to fetch the todos and dispatch actions from a global context.

Clear selection

You are developing a **data-intensive dashboard** where multiple widgets show different visualizations based on API data. Each widget fetches data independently. To optimize performance, you want to prevent each widget from re-fetching the same data multiple times if another widget has already fetched it. What would be an appropriate solution? 1 point

- ○ Use useState in each widget to store its data and fetch it separately.
- ◉ Use useContext to share the fetched data among widgets so that they don't need to fetch the same data multiple times.
- ○ Use useEffect to fetch data only once for each widget and store it in the global window object.
- ○ Use useReducer to manage the data fetching logic inside each widget component.

Clear selection

You are building a React application that fetches data from an external API. Sometimes, the API request fails due to network issues or a server error. How can you handle these **errors gracefully** in your React component? 1 point

- ○ Ignore the error and let the application crash to show a detailed error message.
- ◉ Use a try-catch block around the API call inside useEffect or a callback function.
- ○ Set the component state inside the API call, and if there's an error, ignore the state update.
- ○ Use window.onerror to catch all errors globally.

Clear selection

What is one key advantage of using React Server Components (RSC)? 1 point

- ○ They allow client-side state management using hooks like useReducer.
- ◉ They reduce the amount of JavaScript sent to the client by rendering components on the server.
- ○ They allow direct DOM manipulation on the server.
- ○ They are required for all React applications using Next.js.

Clear selection

Back Submit Clear form

Never submit passwords through Google Forms.

This content is neither created nor endorsed by Google. - Terms of Service - Privacy Policy

Does this form look suspicious? Report

Google Forms